DevCon Lozärn 2026

Local AI – Agentic Coding

Tool-Calls, Workflows und Coding ohne Cloud

Alexander Ramm - Pure Consulting AG

Pure

AUSGANGSPUNKT

Ein Modell, das nur redet, ist erst der halbe Weg.

Spannend wird es, wenn es selbst Werkzeuge aufruft, Code ausführt und das Ergebnis prüft.

UNSER BEISPIEL FÜR HEUTE

«Suche auf meinem Rechner den letzten Quartalsbericht und kopiere ihn auf meinen USB-Stick.»

SO SIEHT EIN TOOL-CALL AUS

Das Modell ruft nichts auf – es beschreibt den Aufruf

1 · Werkzeug anbieten

```
{"name": "copy_to_usb",
 "description":
   "Kopiert eine Datei
    auf den USB-Stick",
 "parameters": {
   "path": {"type": "string"}
 }}
```

2 · Modell antwortet

```
{"role": "assistant",
 "tool_calls": [{
   "function": {
     "name": "copy_to_usb",
     "arguments": {
       "path": "~/Berichte/
                Q3-2026.pdf"
 }}}]}
```

3 · Unser Code führt aus

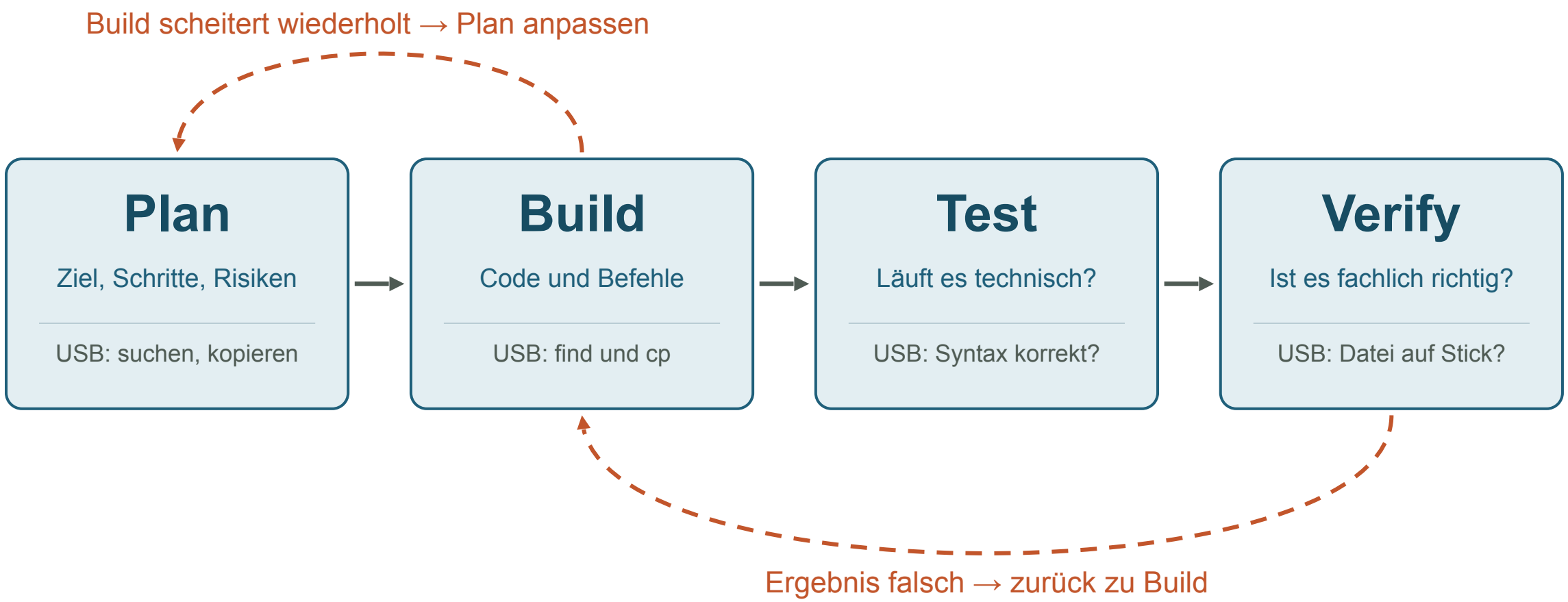
```
{"role": "tool",
 "content":
   "ok: 2,4 MB nach
    /Volumes/STICK
    kopiert"}


→ zurück ans Modell
```

Bei kleinen Modellen: reduzierte Tool-Calls (`read`, `write`, `shell`!)

DAS PRINZIP

Plan → Build → Test → Verify



DAS PRINZIP

Test fragt: läuft es? Verify fragt: stimmt es?



Test – technisch

Syntax korrekt, Befehl läuft durch, Unit-Tests grün.



Verify – fachlich

Das Ziel ist erreicht: die richtige Datei am richtigen Ort, die Seite tut, was sie soll.

Derselbe Zyklus beim USB-Stick und bei einer WebApp

Schritt	Quartalsbericht auf USB-Stick	Kleine WebApp
Plan	Bericht finden, Stick erkennen, kopieren	Seite, Komponenten, Datenquelle
Build	`find`, `cp`	HTML, CSS und JS schreiben
Test	Läuft der Befehl? Syntax korrekt?	Unit-Tests, automatisierter Browsertest
Verify	Datei auf dem Stick, richtige Version?	Im Browser öffnen und durchklicken
← Build	Falscher Pfad → neuer Befehl	Layout kaputt → Code korrigieren
← Plan	Stick nicht gemountet → anderer Plan	Falsche Architektur → neu planen

Beide durchlaufen denselben Zyklus – mit unterschiedlicher Fehlerquote

Schritt	Kleines Modell (Laptop)	Grosses Modell (Server)
Plan	Gut bei kurzen, klaren Aufgaben, verliert bei langen Plänen den Faden	Zerlegt auch komplexe Aufgaben sinnvoll
Build	Solide bei kleinen Schritten, Tool-Calls öfter fehlerhaft	Längere Code-Stücke, Tool-Calls meist sauber
Test	Kann Tests ausführen, deutet Fehlermeldungen schwächer	Findet aus Fehlermeldungen öfter die Ursache
Verify	Hält falsche Ergebnisse eher für richtig	Auch hier gilt: Selbstkontrolle ersetzt keine Prüfung

Und wer prüft Verify?

Beim USB-Stick geht Verify nur auf dem Live-System. Genau da wird es heikel.

RISIKO

Gleicher Zyklus, sehr unterschiedliches Risiko

Schritt	USB-Stick (Tool-Agent)	WebApp (Coding-Agent)
Plan	🟡 Welcher Bericht? Welcher Stick?	🟢 Anforderung ist beschrieben
Build	🟢 `find` und `cp` – einfach	🟢 Code entsteht im Projektordner
Test	🟡 Nur Syntax prüfbar, ohne zu kopieren	🟢 Unit-Tests in der Sandbox
Verify	🔴 Nur live: auf dem echten Stick	🟢 Im Browser der Sandbox durchklicken

WIE WIR ABSICHERN

Drei Leitplanken – für kleine und grosse Modelle

Klare Regeln

Erlaubte Werkzeuge und Pfade, Schrittlimit, Rückfrage vor allem, was sich nicht rückgängig machen lässt.

Sandbox

Erst in einer abgeschotteten Umgebung bauen, testen und prüfen. Erst danach live.

Passendes Werkzeug

Wenige, spezialisierte Werkzeuge statt einer offenen Shell.

WIE WIR ABSICHERN

Erst in der Sandbox, dann live



- Modell läuft auf dem Host, der Agent im Container
- Netzwerk: nur ein Proxy zum Modell, sonst nichts

Kleine Modelle brauchen engere Leitplanken

1. Fünf bis sechs Werkzeuge mit strengen Schemas
2. Fester Ablauf statt freier Schleife: Plan → Build → Test → Verify
3. Schrittlimit und sauberer Abbruch
4. Tests als objektiver Prüfer, nicht das Modell selbst

WAS AUF DEM LAPTOP GUT GEHT

Klein, abgeschlossen, sofort prüfbar

Beispiel: eine Feuerwerk-Animation aus einem einzigen Prompt.

Abgeschlossen

Eine Seite, ein Ordner, keine Abhängigkeit zum Live-System.

Sofort sichtbar

Verify heisst hier: hinschauen. Funktioniert es oder nicht?

Fehler kosten nichts

Geht etwas schief, wird die Sandbox einfach neu gestartet.

DEMO

Ein Agent baut eine App – komplett lokal.

Aufzeichnung · Plan → Build → Test → Verify in der Sandbox

Code und Beispiele: https://github.com/pureconsulting-AG/devcon2026





WAS HEUTE GEHT

Speicher entscheidet, ob ein Modell läuft – Bandbreite, wie schnell

Hardware	Speicher für Modelle	Beispielmodell	Reicht für
Laptop	6–16 GB	Qwen 3.5 9B	Tool-Calls, kleiner Code
Desktop-GPU	16–32 GB	Qwen 3.6 35B-A3B 35 Mrd., 3 Mrd. aktiv	Gutes Coding, mehrstufige Agenten
128-GB-Klasse	~100–120 GB	gpt-oss 120B 117 Mrd., 5 Mrd. aktiv	Starkes Reasoning
Workstation	256–512 GB	GLM-5.3-Flash 320 Mrd., 18 Mrd. aktiv	Agentisches Coding nahe der Spitze

WAS HEUTE GEHT

Heute – lokales Spitzenmodell

GLM-5.3-Flash

320 Mrd. Parameter, davon 18 Mrd. aktiv. MIT-Lizenz.

Niveau

Auf der Höhe von Claude Opus 5.0 vom 24. Juli 2026 (im Agentic Coding).

Preis

Läuft ab ~128 GB Speicher. In voller Qualität: Workstation im fünfstelligen Bereich.

WAS BALD GEHT – UNSERE ANNAHME

Das Niveau von heute wandert nach unten



Heute

Laptop: Tool-Calls und kleiner Code. GLM-Niveau nur auf der Workstation.



In 2 Jahren

Gutes Coding auf dem Laptop. GLM-Niveau auf einem bezahlbaren Team-Server.



In 4 Jahren

GLM-5.3-Niveau auf günstiger Hardware. Lokales Agentic Coding als Normalfall.

* Jede Ebene hinkt der nächsthöheren um 1–2 Jahre hinterher: Cloud-Spitze → Workstation → Team-Server → Laptop.

WAS BLEIBT

Bessere Modelle machen weniger Fehler.

Plan → Build → Test → Verify bleibt. Und Verify bleibt Pflicht.

ZUM MITNEHMEN

Drei Regeln

1. Klare Regeln für den Agenten
2. Erst Sandbox, dann live
3. Verify ist Pflicht, nicht optional

Fragen?

Gebt mir gerne Feedback



Github Repo

